

CeTZ Gallery

Diagrams in idiomatic CeTZ 0.5.2, each chosen for a technique worth copying.

[View as PDF](#). [View as HTML](#). [View source](#).

Every figure is a CeTZ redrawing of a TikZ figure from Walmes M. Zeviani's *Tikz Gallery* [1]. Each entry links its original. The redrawings translate the labels from Portuguese to English, use Typst's built-in colors, and differ in detail.

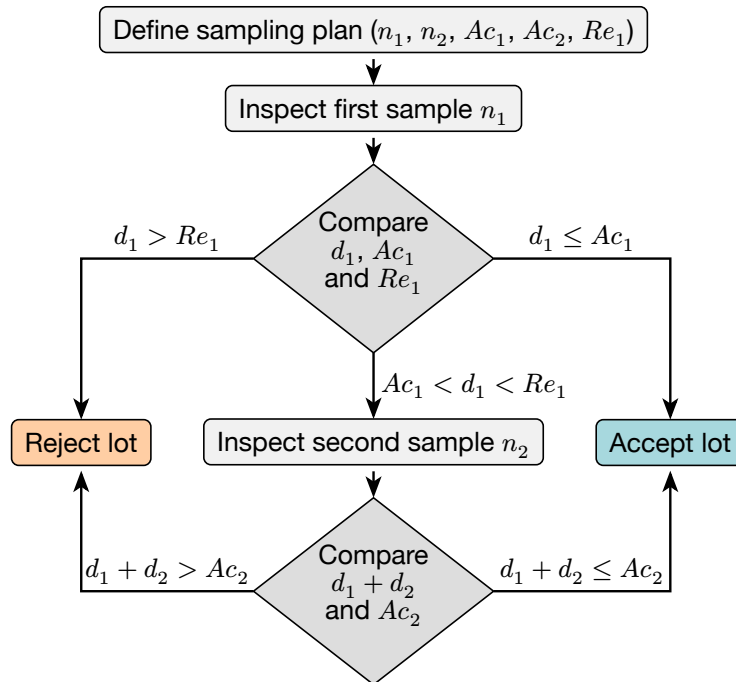
[1] Walmes M. Zeviani, *Tikz Gallery*. <https://github.com/walmes/Tikz>, <http://leg.ufpr.br/~walmes/Tikz/>.

Figures

Sampling-plan flowchart	2
Series-parallel blocks	4
System with inputs and output	5
Overlapping groups	6
Probability tree	8
Cycle on a triangle	9
Density with a shaded tail	10
Factorial-design cube	11
Interval with braces	12
Directory tree	13
Venn partition	15
Floor plan	16
Flowchart shapes	18
Commit graph	20
Mind map	22
Timeline	24
Capillary rise	26
Threshold crossing	28
Hatched intervals	30
Ternary diagram	32
Regression densities in 3D	34
Bias and variance targets	36

Sampling-plan flowchart

Redrawn from [sampling-plan-2.pgff \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. Relative placement from anchors; diamond decisions built from a text box's anchors; elbow branches with labels; two arrows meeting one box.



```

#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// Double-sampling acceptance plan: Ac = acceptance number, Re = rejection number.
#let Ac = $A c$
#let Re = $R e$

#set cetz.canvas({
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)

  // A Typst box gives rounded corners while the anchors stay on its bounding box.
  let step(pos, name, body, fill: silver.lighten(60%), anchor: "north") = content(pos, name:
name,
  anchor: anchor, box(body, fill: fill, stroke: 0.6pt + black, radius: 3pt, inset: 5pt))
  // A diamond whose edge midpoints touch the corners of the text box.
  let decision(pos, name, body) = group(name: name, anchor: "north", {
    content(pos, box(width: 4.5em, align(center, par(leading: 0.35em, body))), name: "text")
    on-layer(-1, line(..("north", "east", "south", "west").map(a => ("text", 200%, "text." +
a)),
      close: true, fill: gray.lighten(60%)))
  })
  let arrow = line.with(stroke: 0.9pt + black, mark: (end: "stealth", fill: black))
  let branch(from, to, label) = {
    let corner = (from, "-", to)
    arrow(from, corner, to)
    content((from, 50%, corner), label, anchor: "south", padding: 0.1)
  }

  let gap = 0.35
  step((0, 0), "plan", [Define sampling plan ($n_1$, $n_2$, $Ac_1$, $Ac_2$, $Re_1$)])
  step((rel: (0, -gap), to: "plan.south"), "ins1", [Inspect first sample $n_1$])
  decision((rel: (0, -gap), to: "ins1.south"), "d1", [Compare $d_1$, $Ac_1$ and $Re_1$])
  step((rel: (0, -2.5 * gap), to: "d1.south"), "ins2", [Inspect second sample $n_2$])

```

```

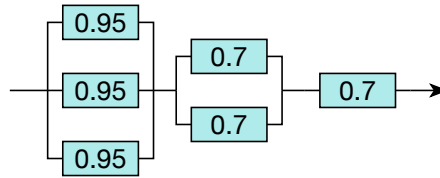
decision((rel: (0, -gap), to: "ins2.south"), "d2", [Compare  $d_1 + d_2$  and  $Ac_2$ ])
step((rel: (2 * gap, 0), to: "ins2.east"), "accept", [Accept lot],
  fill: eastern.lighten(60%), anchor: "west")
step((rel: (-2 * gap, 0), to: "ins2.west"), "reject", [Reject lot],
  fill: orange.lighten(60%), anchor: "east")

arrow("plan", "ins1")
arrow("ins1", "d1")
arrow("d1", "ins2")
content(("d1.south", 50%, "ins2.north"),  $Ac_1 < d_1 < Re_1$ , anchor: "west", padding: 0.1)
arrow("ins2", "d2")
branch("d1.east", "accept.north",  $d_1 \leq Ac_1$ )
branch("d1.west", "reject.north",  $d_1 > Re_1$ )
branch("d2.east", "accept.south",  $d_1 + d_2 \leq Ac_2$ )
branch("d2.west", "reject.south",  $d_1 + d_2 > Ac_2$ )
})

```

Series-parallel blocks

Redrawn from [circuitos_mistos1.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. A data-driven loop; forks and joins with perpendicular coordinates; wires drawn behind the boxes with `on-layer`.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

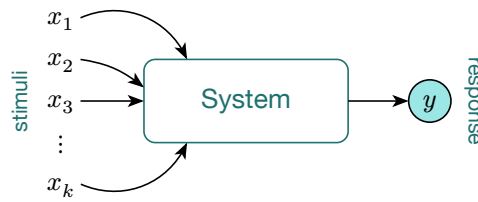
// Series-parallel reliability block diagram: each stage holds redundant
// components in parallel, and the stages are connected in series.
#cezt.canvas({
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)

  let stages = ((0.95, 3), (0.7, 2), (0.7, 1)) // (reliability, count)
  let (dx, dy, w, h) = (1.7, 0.9, 1, 0.45)

  let prev = (rel: (-0.5, 0), to: "s0.west") // resolved when drawn, after "s0" exists
  for (i, (p, n)) in stages.enumerate() {
    let s = "s" + str(i)
    // The x-padding puts the group's west/east anchors on the fork and join buses.
    group(name: s, padding: (x: 0.2), {
      for k in range(n) {
        rect((i * dx, ((n - 1) / 2 - k) * dy), (rel: (w, h)), anchor: "center",
          fill: teal.lighten(60%), name: str(k))
        content(str(k), [#p])
      }
    })
    // Wires pass straight through the boxes, so draw them behind.
    on-layer(-1, {
      let (fork, join) = (s + ".west", s + ".east")
      line(prev, fork)
      for k in range(n) {
        let b = s + "." + str(k)
        line(fork, (fork, "|-", b), (join, "|-", b), join)
      }
    })
    prev = s + ".east"
  }
  line(prev, (rel: (0.5, 0)), mark: (end: "stealth", fill: black))
})
```

System with inputs and output

Redrawn from [reg_sistema.pgf](#) (png) in Walmes M. Zeviani's *Tikz Gallery*. Curved connectors ending at border anchors given as angles; a circle joined to a box; rotated captions.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#let accent = teal.darken(45%)

#cetx.canvas({
  import cetx.draw: *
  set-style(
    stroke: 0.6pt + black,
    mark: (end: "stealth", fill: black, scale: 0.8),
    content: (padding: 0.08),
  )

  // Cubic curve leaving `a` and entering `b` at `angle` to the chord, like TikZ `bend left`.
  let bend(a, b, angle) = bezier(a, b, (a, 40%, angle, b), (b, 40%, -angle, a))

  rect((0, 0), (rel: (2.7, 1.1)), radius: 0.15, stroke: accent, name: "sys")
  content("sys", text(accent)[System])

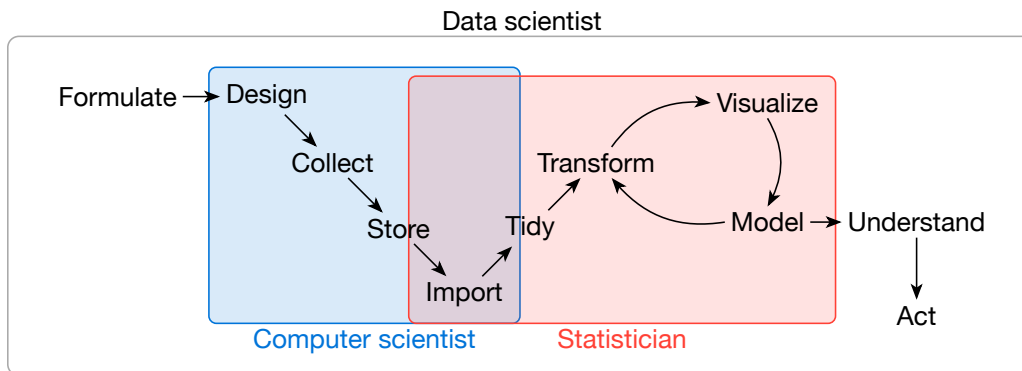
  let inputs = (x1: $x_1$, x2: $x_2$, x3: $x_3$, dots: $dots.v$, xk: $x_k$)
  for (i, (name, x)) in inputs.pairs().enumerate() {
    content((rel: (-1.1, (2 - i) * 0.55), to: "sys.west"), x, name: name)
  }
  bend("x1.east", (name: "sys", anchor: 120deg), 45deg)
  bend("x2.east", (name: "sys", anchor: 162deg), 15deg)
  line("x3", "sys")
  bend("xk.east", (name: "sys", anchor: 240deg), -45deg)

  circle((rel: (0.8, 0), to: "sys.east"), radius: 0.28, anchor: "west",
    fill: teal.lighten(50%), name: "y")
  content("y", $y$)
  line("sys", "y")

  set-style(content: (wrap: text.with(8pt, accent)))
  content((rel: (-0.1, 0), to: "x3.west"), [stimuli], angle: 90deg, anchor: "south")
  content((rel: (0.1, 0), to: "y.east"), [response], angle: -90deg, anchor: "south")
})
```

Overlapping groups

Redrawn from [data-science-workflow.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. Translucent frames around groups of nodes with `rect-around`; a cycle of bent arrows; placement by polar offsets.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetx.canvas({
  import cetx.draw: *
  set-style(
    stroke: 0.6pt + black,
    mark: (end: "stealth", fill: black, scale: 0.8),
    content: (padding: 0.08),
  )

  // Bare element names make `line` stop at both borders.
  let step(from, name, offset) = {
    content((rel: offset, to: from), name, name: name)
    line(from, name)
  }

  // Cubic curve leaving `a` and entering `b` at `angle` to the chord, like TikZ `bend left`.
  let bend(a, b, angle) = bezier(a, b, (a, 40%, angle, b), (b, 40%, -angle, a))

  content((0, 0), "Formulate", name: "Formulate")
  let prev = "Formulate"
  for (name, offset) in (
    ("Design", (0deg, 2)), ("Collect", (-45deg, 1.25)), ("Store", (-45deg, 1.25)),
    ("Import", (-45deg, 1.25)), ("Tidy", (45deg, 1.25)), ("Transform", (45deg, 1.25)),
  ) {
    step(prev, name, offset)
    prev = name
  }

  content((rel: (2.3, 0.8), to: "Transform"), "Visualize", name: "Visualize")
  content((rel: (2.3, -0.8), to: "Transform"), "Model", name: "Model")
  bend((name: "Transform", anchor: 45deg), "Visualize.west", 30deg)
  bend("Visualize.south", "Model.north", 30deg)
  bend("Model.west", (name: "Transform", anchor: -45deg), 30deg)

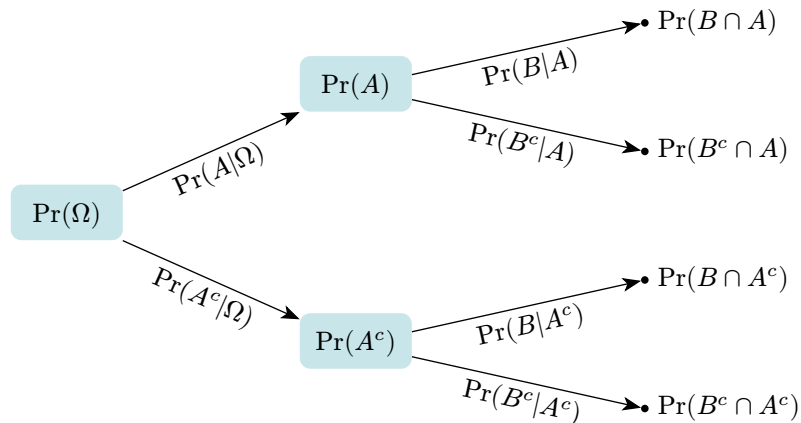
  step("Model", "Understand", (0deg, 2))
  step("Understand", "Act", (-90deg, 1.25))

  on-layer(-1, {
    for (members, hue, caption) in (
      (("Design", "Import"), blue, [Computer scientist]),
      (("Import", "Visualize"), red, [Statistician]),
    ) {
      rect-around(..members, padding: 0.15, radius: 0.1, name: "group",
        stroke: hue, fill: hue.transparentize(85%))
    }
  })
}
```

```
    content("group.south", text(hue, caption), anchor: "north")
  }
  rect-around("Formulate", "Import", "Understand", "Act", padding: 0.6, radius: 0.15,
    stroke: gray, name: "all")
  content("all.north", [Data scientist], anchor: "south")
})
})
```

Probability tree

Redrawn from [probtree_scheme.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. `cetz.tree` growing to the right; custom node and edge callbacks; labels along the edges.



```

#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetz.canvas({
  import cetz.draw: *
  set-style(
    stroke: .5pt + black,
    mark: (end: "stealth", fill: black),
    content: (padding: .12),
  )

  let draw-node(n) = if n.depth < 2 {
    let bag = box.with(fill: eastern.lighten(75%), radius: 4pt, inset: 7pt, width: 4.2em)
    content((0, 0), padding: 0, bag(align(center, $Pr(#n.content)$)))
  } else {
    circle((0, 0), radius: .05, fill: black, stroke: none)
  }

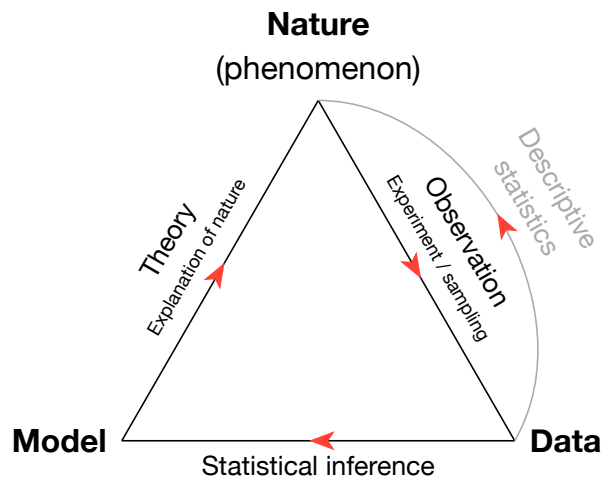
  let draw-edge(parent, child) = {
    let (a, b) = (parent.content, child.content)
    line(parent.group-name, child.group-name, name: "e")
    content("e.mid", $Pr(#b|#a)$, angle: "e.end", anchor: "north")
    // Only the edge callback sees both events, so it also labels the leaf.
    if child.depth == 2 {
      content(child.group-name + ".east", $Pr(#b inter #a)$, anchor: "west")
    }
  }

  // With direction "right", children are listed bottom to top.
  cetz.tree.tree(
    ($Omega$, ($A^c$, $B^c$, $B$), ($A$, $B^c$, $B$)),
    direction: "right", grow: 3, spread: 1.6,
    draw-node: draw-node, draw-edge: draw-edge,
  )
})

```

Cycle on a triangle

Redrawn from [knowledge-cycle.png](#) in Walmes M. Zeviani's *Tikz Gallery*. Polar coordinates; arrowheads in the middle of each edge; text rotated along edges; a Bézier curve with polar control points.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#let note(title, sub) = align(center)[#title \ #text(size: 7pt, sub)]

#cetz.canvas({
  import cetz.draw: *
  set-style(
    stroke: .6pt + black,
    // Mid-path arrowhead; shorten-to: none keeps CeTZ from cutting the path at the mark.
    mark: (end: "stealth", pos: 50%, anchor: "center", shorten-to: none,
      scale: 1.4, fill: red, stroke: red),
    content: (padding: .2),
  )

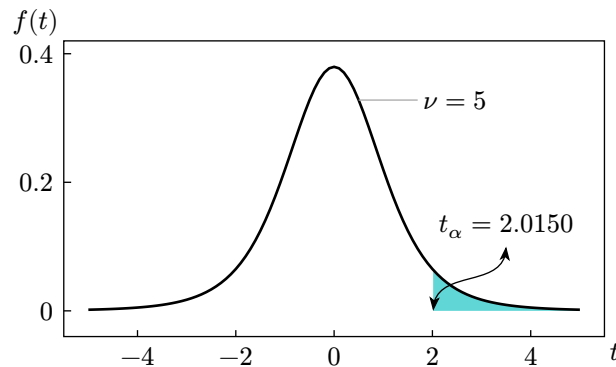
  anchor("nature", (90deg, 3))
  anchor("model", (210deg, 3))
  anchor("data", (330deg, 3))
  content("nature", text(12pt, align(center)[*Nature* \ (phenomenon)]), anchor: "south")
  content("model", text(12pt)[*Model*], anchor: "east")
  content("data", text(12pt)[*Data*], anchor: "west")

  let edge(a, b, label) = {
    line(a, b, name: "e")
    content("e.mid", label, angle: "e.end", anchor: "south")
  }
  edge("model", "nature", note[Theory][Explanation of nature])
  edge("nature", "data", note[Observation][Experiment / sampling])
  line("data", "model", name: "inference")
  content("inference.mid", [Statistical inference], anchor: "north")

  bezier("data", "nature", (rel: (60deg, 2), to: "data"), (rel: (0deg, 2), to: "nature"),
    name: "arc", stroke: gray, mark: (fill: red, stroke: red))
  content("arc.mid", angle: -60deg, anchor: "south",
    text(fill: gray, align(center)[Descriptive \ statistics]))
})
```

Density with a shaded tail

Redrawn from [dist_t_quantil.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. A plot in data units via `scale`; a sampled curve; a filled tail; tick loops; a curved callout.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetx.canvas({
  import cetx.draw: *
  set-style(stroke: (paint: black, thickness: .5pt), mark: (fill: black, scale: .7))
  // Data coordinates (t, f(t)); text, strokes and marks keep their size.
  scale(x: .65, y: 8.5)

  // Student t density for ν = 5: Γ(3) / (√(5π) Γ(5/2)) · (1 + t²/5)^(-3).
  let pdf(t) = 8 / (3 * calc.pi * calc.sqrt(5)) * calc.pow(1 + t * t / 5, -3)
  let curve(a, b) = range(101).map(i => a + (b - a) * i / 100).map(t => (t, pdf(t)))
  let q = 2.015

  line((q, 0), ..curve(q, 5), (5, 0), close: true, stroke: none,
    fill: teal.lighten(20%))
  line(..curve(-5, 5), stroke: (thickness: 1pt))

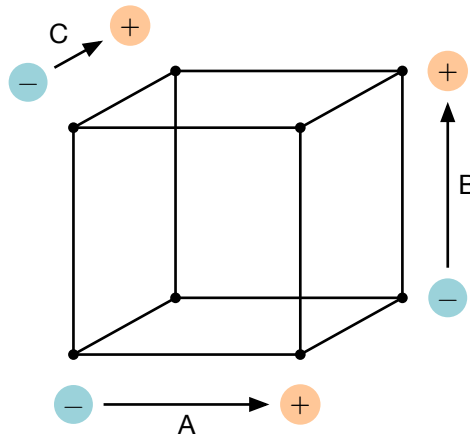
  rect((-5.5, -.04), (5.5, .42), name: "frame")
  for x in (-4, -2, 0, 2, 4) {
    let p = ((x, 0), "|-", "frame.south")
    line(p, (rel: (0, .01)))
    content(p, $x$, anchor: "north", padding: 4pt)
  }
  for y in (0, .2, .4) {
    let p = ("frame.west", "|-", (0, y))
    line(p, (rel: (.15, 0)))
    content(p, $y$, anchor: "east", padding: 4pt)
  }
  content("frame.south-east", $t$, anchor: "north-west", padding: 2pt)
  content("frame.north-west", $f(t)$, anchor: "south-east", padding: 2pt)

  line((.5, pdf(.5)), (rel: (1.2, 0)), stroke: gray, name: "leader")
  content("leader.end", $nu = 5$, anchor: "west", padding: 2pt)

  bezier((q, 0), (3.5, .1), (q, .07), (3.5, .03),
    mark: (start: "stealth", end: "stealth"), name: "callout")
  content("callout.end", $t_alpha = 2.0150$, anchor: "south", padding: 3pt)
})
```

Factorial-design cube

Redrawn from [cubos-fatoriais-2a3.pg](#) (png) in Walms M. Zeviani's *Tikz Gallery*. An oblique 3D view from the canvas's z vector; a loop over vertices; framed circle markers joined by arrows.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

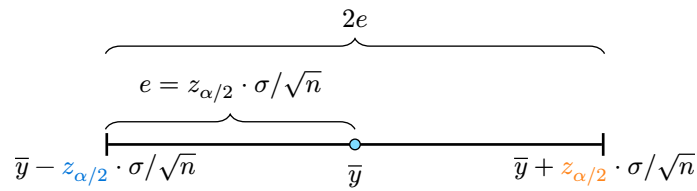
// 2^3 factorial design: each vertex is one run, each axis one factor at levels -/+.
// Oblique projection: the canvas draws the z unit vector as (.45, .25).
#cezt.canvas(length: 3cm, z: (.45, .25), {
  import cetz.draw: *
  set-style(stroke: black, mark: (fill: black))

  for x in (0, 1) { for y in (0, 1) { for z in (0, 1) {
    let p = (x, y, z)
    if x == 0 { line(p, (rel: (1, 0, 0))) }
    if y == 0 { line(p, (rel: (0, 1, 0))) }
    if z == 0 { line(p, (rel: (0, 0, 1))) }
    circle(p, radius: 2pt, fill: black, stroke: none)
  } } }

  // Level discs beside the low and high vertex; a line between named
  // elements stops at their borders.
  set-style(content: (frame: "circle", stroke: none, padding: 1.5pt))
  let factor(id, low, high, offset, anchor) = {
    content((rel: offset, to: low), $-$, fill: eastern.transparentize(55%), name: id + "-")
    content((rel: offset, to: high), $+$, fill: orange.transparentize(55%), name: id + "+")
    // The invisible start mark makes `offset` leave a gap at both ends.
    line(id + "-", id + "+", mark: (start: (symbol: "|", stroke: none), end: ">", offset: 4pt))
    content((id + "-", 50%, id + "+"), id, anchor: anchor, frame: none, padding: 4pt)
  }
  factor("A", (0, 0, 0), (1, 0, 0), (0, -.22), "north")
  factor("B", (1, 0, 1), (1, 1, 1), (.2, 0), "west")
  factor("C", (0, 1, 0), (0, 1, 1), (-.2, .2), "south-east")
})
```

Interval with braces

Redrawn from [confidence-interval-mean-components.pgfc \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. flat-brace with labels; interval end marks; colored parts of math labels.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// Margin of error; `fill` colors the z.
#let half(fill) = $#text(fill: fill, $z_{(\alpha\!/2)}$) dot sigma \!/\! sqrt(n)$

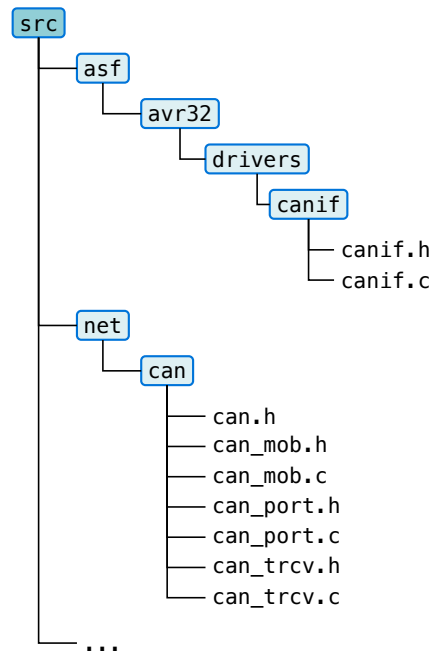
#cezt.canvas({
  import cetz.draw: *
  import cetz.decorations: flat-brace
  set-style(
    stroke: (paint: black, thickness: .5pt),
    content: (padding: .15),
    flat-brace: (amplitude: .2, content-offset: 0),
  )

  line((-3.3, 0), (3.3, 0), name: "ci", stroke: (thickness: 1pt), mark: (symbol: "|",
width: .3))
  circle("ci.50%", radius: .07, fill: aqua)
  content("ci.50%", $overline(y)$, anchor: "north", padding: .25)
  content("ci.start", $overline(y) - #half(blue)$, anchor: "north")
  content("ci.end", $overline(y) + #half(orange)$, anchor: "north")

  flat-brace((rel: (0, .2), to: "ci.start"), (rel: (0, .2), to: "ci.50%"), name: "e")
  content("e.content", $e = #half(black)$, anchor: "south")
  flat-brace((rel: (0, 1.2), to: "ci.start"), (rel: (0, 1.2), to: "ci.end"), name: "2e")
  content("2e.content", $2e$, anchor: "south")
})
```

Directory tree

Redrawn from [file_tree.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. A tree drawn from a flat data list; elbow connectors from each parent's south to each child's west.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// (depth, name) in `tree` order; an entry is a directory if the next one is deeper.
#let entries = (
  (0, "src"),
  (1, "asf"), (2, "avr32"), (3, "drivers"), (4, "canif"),
  (5, "canif.h"), (5, "canif.c"),
  (1, "net"), (2, "can"),
  ..("can.h", "can_mob.h", "can_mob.c", "can_port.h", "can_port.c", "can_trcv.h", "can_trcv.c")
  .map(file => (3, file)),
  (1, "..."),
)

#cetz.canvas({
  import cetz.draw: *
  set-style(stroke: (paint: black, thickness: .6pt))

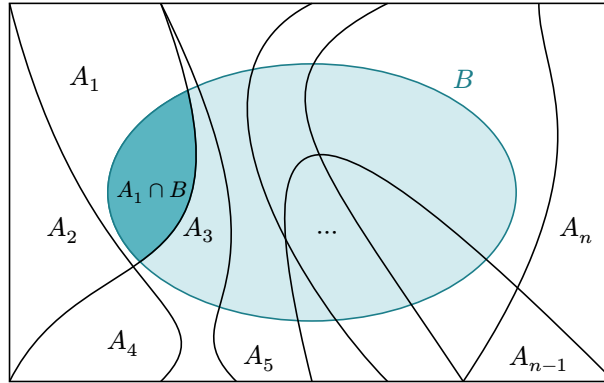
  let y = 0
  let prev-depth = 0
  for (i, (depth, name)) in entries.enumerate() {
    let dir = entries.at(i + 1, default: (0,)).at(0) > depth
    y -= if depth == prev-depth { .4 } else { .6 }
    prev-depth = depth

    // Reused names: "d2" is the latest depth-2 node, i.e. the parent of each new depth-3 node.
    let node = "d" + str(depth)
    content((depth * .85, y), name: node, anchor: "west", box(
      raw(name), inset: 2.5pt, radius: 2pt,
      fill: if depth == 0 { eastern.lighten(50%) } else if dir { eastern.lighten(85%) },
      stroke: if dir { blue + .8pt },
    ))
  }
  if depth > 0 {
    let parent = "d" + str(depth - 1) + ".south"
    line(parent, (parent, "|-", node + ".west"), node + ".west")
  }
})
```

```
}  
})
```

Venn partition

Redrawn from [diagrama_venn4.pg](#) (png) in Walmes M. Zeviani's *Tikz Gallery*. `boolean(op: "intersection")` of a closed merge-path and a named ellipse, in place of clipping; loops for the partition curves and labels.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

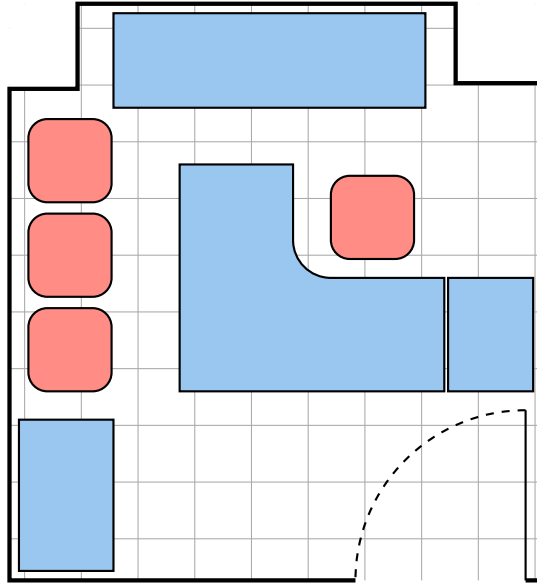
#set cetz.canvas({
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)
  let hue = eastern
  let border-a1 = bezier.with((0, 0), (2, 5), (1, 2), (3.5, 1))

  circle((4, 2.5), radius: (2.7, 1.7), fill: hue.lighten(80%), stroke: hue.darken(20%), name:
  "b")
  // The black border of A1 is drawn over the stroke of the intersection.
  boolean(merge-path(close: true, { border-a1(); line((0, 5)) }, "b", op: "intersection",
    fill: hue.lighten(25%), stroke: hue.darken(20%))
  rect((0, 0), (8, 5))
  border-a1()
  // Boundaries between the remaining events, as (start, end, control 1, control 2).
  for (a, b, c1, c2) in (
    ((0, 5), (2, 0), (1, 1), (3, 1)),
    ((3, 0), (2, 5), (2, 1), (4, 1)),
    ((4, 5), (5, 0), (2, 4), (4, 1)),
    ((4, 0), (8, 0), (3, 4), (4, 4)),
    ((5, 5), (6, 0), (3, 4), (4, 3)),
    ((7, 5), (6, 0), (7, 4), (8, 3)),
  ) { bezier(a, b, c1, c2) }

  content((1.88, 2.5), text(8.5pt, $A_1 inter B))
  content((6, 4), text(hue.darken(20%), $B))
  for (pos, label) in (
    ((1, 4), $A_1$), ((0.7, 2), $A_2$), ((2.5, 2), $A_3$), ((1.5, 0.5), $A_4$),
    ((3.3, 0.3), $A_5$), ((4.2, 2), $dots.c$), ((7, 0.3), $A_{n-1}$), ((7.5, 2), $A_n$),
  ) { content(pos, label) }
})
```

Floor plan

Redrawn from [floor-plan.png](#) in Walmes M. Zeviani's *Tikz Gallery*. Walls from a list of `rel:` moves; `boolean(op: "difference")` hides the grid outside the room; a door swing `arc` from an anchor; furniture as rotated groups with a scoped style.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetx.canvas(length: 2.5mm, {
  import cetx.draw: *
  set-style(stroke: 0.8pt + black)

  // Walls run counterclockwise from the door hinge to the far door jamb.
  let hinge = (27.3, 0)
  let walls = (hinge, (rel: (1, 0)), (rel: (0, 26.3)), (rel: (-4.7, 0)), (rel: (0, 4.2)),
    (rel: (-20, 0)), (rel: (0, -4.5)), (rel: (-3.6, 0)), (rel: (0, -26)), (rel: (18.3, 0)))
  let bounds = ((0, 0), (28.3, 30.5))

  grid(..bounds, step: 3, shift: (0.8, 2.2), stroke: 0.4pt + gray)
  boolean(rect(..bounds), line(..walls, close: true), op: "difference", fill: white, stroke:
none)
  line(..walls, stroke: 1.6pt)

  line(hinge, (rel: (0, 9)), name: "leaf")
  arc("leaf.end", start: 90deg, delta: 90deg, radius: 9, stroke: (dash: "dashed"))

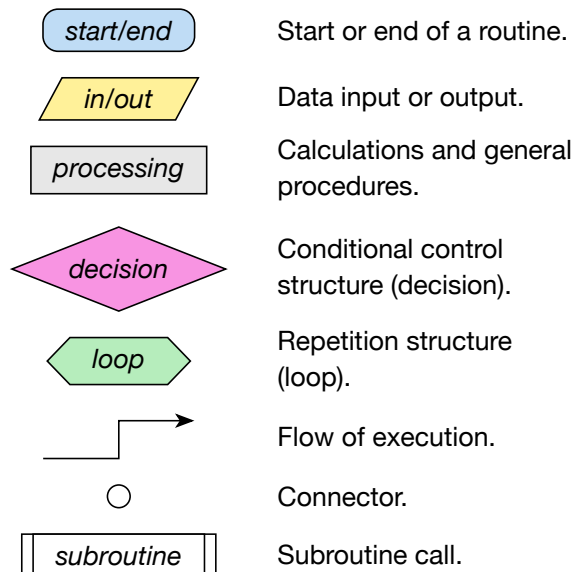
  let piece(origin, angle, body) = group({
    set-style(fill: blue.lighten(60%)); translate(origin); rotate(angle); body })
  piece((0.5, 8.5), -90deg, rect((0, 0), (8, 5))) // bookcase
  piece((22, 25), 90deg, rect((0, 0), (5, 16.5))) // low cupboard
  piece((9, 22), -90deg, {
    // desk and drawer unit
    merge-path(close: true, {
      line((0, 0), (rel: (12, 0)), (rel: (0, 14)), (rel: (-6, 0)), (rel: (0, -6)))
      arc((0, 14), start: 0deg, delta: -90deg, radius: 2)
      line((0, 14), (0, 6))
    })
    rect((6, 14.2), (rel: (6, 4.5)))
  })

  for pos in ((17, 17), (1, 20), (1, 15), (1, 10)) {
    rect(pos, (rel: (4.4, 4.4)), radius: 1, fill: red.lighten(40%))
  }
})
```

```
}  
})
```

Flowchart shapes

Redrawn from [fluxograms.pgfr \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*, which is based on [an answer by Claudio Fiandrino](#) on TeX Stack Exchange (CC BY-SA 3.0). Shapes as small functions that draw their outline from the label's anchors (rect-around, parallelogram, hexagon, diamond); rows from a data list, stacked with "|-" coordinates.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)
// Based on https://tex.stackexchange.com/a/87956 by Claudio Fiandrino, CC BY-SA 3.0.

#cetx.canvas({
  import cetx.draw: *
  set-style(stroke: 0.6pt + black)

  // A shape is its label "t" plus an outline drawn around the label's anchors.
  let node(body, outline, pad-x: 0.3) = pos => {
    content(pos, emph(body), name: "t", padding: (x: pad-x, y: 0.15))
    on-layer(-1, outline)
  }
  let chamfer = 0.3
  let rows = (
    (node([start/end], rect-around("t", radius: 0.2, fill: blue.lighten(75%))),
      [Start or end of a routine.]),
    (node([in/out], line((rel: (-chamfer, 0), to: "t.south-west"), "t.south-east",
      (rel: (chamfer, 0), to: "t.north-east"), "t.north-west", close: true,
      fill: yellow.lighten(60%))),
      [Data input or output.]),
    (node([processing], rect-around("t", fill: silver.lighten(30%))),
      [Calculations and general \ procedures.]),
    // Vertices at twice the label's half-size put the label's corners on the edges.
    (node([decision], line(..("north", "east", "south", "west").map(a => ("t", 200%, "t." + a)),
      close: true, fill: fuchsia.lighten(55%)), pad-x: 0.05),
      [Conditional control \ structure (decision).]),
    (node([loop], line((rel: (-chamfer, 0), to: "t.west"), "t.north-west", "t.north-east",
      (rel: (chamfer, 0), to: "t.east"), "t.south-east", "t.south-west", close: true,
      fill: green.lighten(65%))),
      [Repetition structure \ (loop).]),
    (pos => line((rel: (-1, -0.25), to: pos), (rel: (1, 0)), (rel: (0, 0.5)), (rel: (1, 0))),
      mark: (end: "stealth", fill: black)),
      [Flow of execution.]),
    (pos => circle(pos, radius: 0.15), [Connector.]),
    // The side bars sit outside the label box, as in TikZ's `predefined process`.
  )
})
```

```

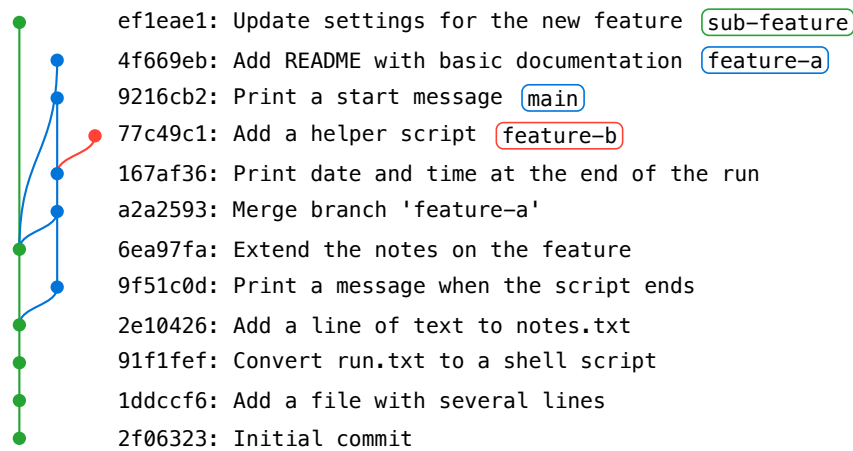
(node([subroutine], {
  rect-around("t", padding: (x: 0.15))
  rect-around("t", fill: white)
}), [Subroutine call.]),
)

// Shapes share a centre line; each row starts below the lower of the previous shape and text.
let text-x = 2.1
for (i, (shape, desc)) in rows.enumerate() {
  let top = if i == 0 { (0, 0) } else { ((0, 0), "|-", (rel: (0, -0.35), to: "row" + str(i -
1) + ".south")) }
  group(name: "row" + str(i), {
    group(name: "shape", anchor: "north", shape(top))
    content(((text-x, 0), "|-", "shape"), desc, anchor: "west")
  })
}
})

```

Commit graph

Redrawn from [git_log.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*, which is based on [an answer by ph0t0nix](#) on TeX Stack Exchange (CC BY-SA 3.0). Layout from a data list of commits, lanes and parents; bezier edges that leave and enter vertically; messages and branch tags placed with perpendicular and relative coordinates.



```

#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)
// Based on https://tex.stackexchange.com/a/262163 by ph0t0nix, CC BY-SA 3.0.

// Newest first, as in `git log --graph`: (hash, lane, parents, message, branch tag).
#let commits = (
  ("ef1eae1", 0, ("6ea97fa",), "Update settings for the new feature", "sub-feature"),
  ("4f669eb", 1, ("6ea97fa",), "Add README with basic documentation", "feature-a"),
  ("9216cb2", 1, ("167af36",), "Print a start message", "main"),
  ("77c49c1", 2, ("167af36",), "Add a helper script", "feature-b"),
  ("167af36", 1, ("a2a2593",), "Print date and time at the end of the run", none),
  ("a2a2593", 1, ("9f51c0d", "6ea97fa"), "Merge branch 'feature-a'", none),
  ("6ea97fa", 0, ("2e10426",), "Extend the notes on the feature", none),
  ("9f51c0d", 1, ("2e10426",), "Print a message when the script ends", none),
  ("2e10426", 0, ("91f1fef",), "Add a line of text to notes.txt", none),
  ("91f1fef", 0, ("1ddccf6",), "Convert run.txt to a shell script", none),
  ("1ddccf6", 0, ("2f06323",), "Add a file with several lines", none),
  ("2f06323", 0, (), "Initial commit", none),
)
#let lanes = (green.darken(20%), blue, red)

#set cetz.canvas({
  import cetz.draw: *

  for (row, (id, lane, _, msg, tag)) in commits.enumerate() {
    let hue = lanes.at(lane)
    circle((lane * 0.5, -row * 0.5), radius: 0.07, fill: hue, stroke: hue, name: id)
    // Messages share a column right of the widest lane, so no edge runs through them.
    content(((1.3, 0), "|-", id), raw(id + ": " + msg), anchor: "west", name: id + "-msg")
    if tag != none {
      content((rel: (0.25, 0), to: id + "-msg.east"), anchor: "west",
        box(raw(tag), stroke: 0.6pt + hue, radius: 3pt, inset: 2pt))
    }
  }
})

// Each edge leaves the parent and enters the child vertically, in the child's lane colour.
on-layer(-1, for (id, lane, parents, ..) in commits {
  for parent in parents {
    let mid = (parent, 50%, id)
    bezier(parent, id, (parent, "|-", mid), (id, "|-", mid), stroke: 0.8pt + lanes.at(lane))
  }
})
  
```

```
}  
}
```

Mind map

Redrawn from [mindmapImovel.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. Children at polar positions from a loop; tapered connectors as a merge-path of two bezier curves; colors from `color.mix`; `gradient.linear` fills behind the circles.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetz.canvas({
  import cetz.draw: *
  set-style(stroke: none)

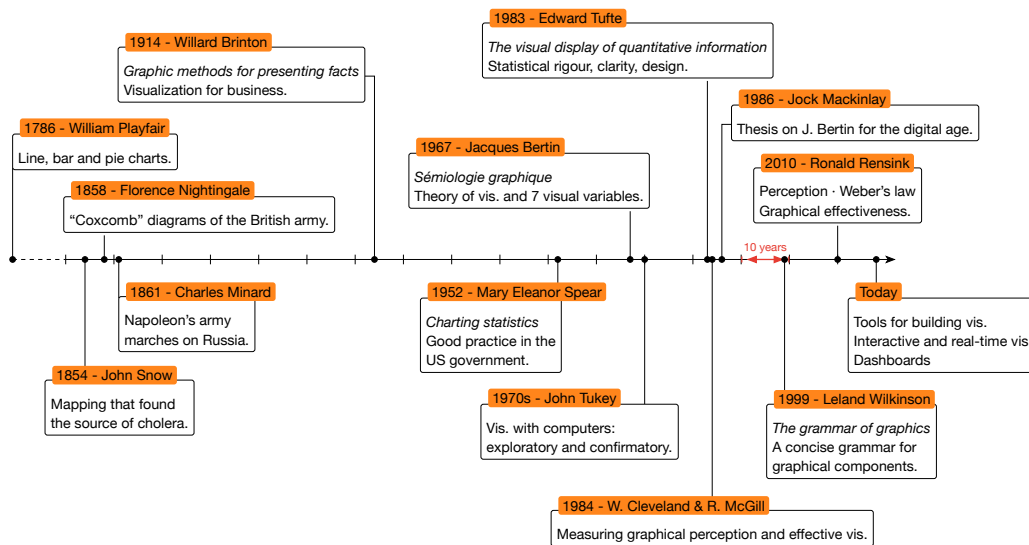
  let (R, r, d) = (1.7, 1.1, 4.4)
  let root = teal.darken(40%)
  let concept(pos, radius, fill, body, ink: white) = {
    circle(pos, radius: radius, fill: fill)
    content(pos, align(center, text(ink, body)))
  }
  // Tapered bar from the root towards `a` with waist 2w; Typst gradient angles run clockwise.
  let bar(a, fill, w: 0.08) = {
    let side(s) = (
      (a + s * 20deg, R),
      (rel: (a + 180deg - s * 30deg, r), to: (a, d)),
      (rel: (a + s * 90deg, w), to: (a, R + 0.7)),
      (rel: (a + s * 90deg, w), to: (a, d - r - 0.6)),
    )
    let (p, q, c1, c2) = side(-1)
    merge-path(close: true, fill: gradient.linear(root, fill, angle: -a), {
      bezier(..side(1))
      bezier(q, p, c2, c1)
    })
  }

  concept((0, 0), R, root, text(13pt)[Sale price \ of a property])
  for (i, (hue, ink, body)) in (
    (white, black, [Area (m²), \ Bedrooms, \ Garage, \ Pool]),
    (blue, white, [Age, \ Condition, \ Finish]),
    (orange, black, [Location, \ Access, \ Security]),
    (purple, white, [Economic \ indicators]),
  ).enumerate() {
    let a = 180deg - i * 60deg
    let fill = color.mix((root, 60%), (hue, 40%))
    on-layer(-1, bar(a, fill))
    concept((a, d), r, fill, ink: ink, body)
  }
})
```

```
}  
})
```

Timeline

Redrawn from [timeline_dataviz.png](#) in Walmes M. Zeviani's *Tikz Gallery*. A data-driven callout helper placed by an angle anchor; leaders with "|-" coordinates; tag tabs relative to a box corner; a year-to-position mapping function.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#set cetz.canvas(length: 1.2cm, {
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)

  // One unit per decade from 1850; the dashed stub stands in for the years back to 1786.
  let x(year) = calc.max((year - 1850) / 10, -1.1)
  line((-1.1, 0), (0, 0), stroke: (dash: "dashed"))
  line((0, 0), (17.2, 0), mark: (end: "stealth", fill: black))
  for i in range(17) { line((i, -0.1), (i, 0.1)) }
  let rule = red.darken(10%)
  line((14, 0), (15, 0), name: "scale", stroke: rule + 0.9pt,
    mark: (symbol: ("|", ">"), fill: rule))
  content("scale.mid", text(8pt, rule)[10 years], anchor: "south", padding: 0.12)

  // The box's border point at angle `at` lies at `offset` from the event.
  let callout(year, offset, at, title, body) = {
    let p = (x(year), 0)
    let a = (name: "box", anchor: at)
    content((rel: offset, to: p), name: "box", anchor: at,
      box(fill: white, stroke: 0.6pt, radius: 2pt, inset: (x: 4pt, top: 13pt, bottom: 5pt),
body))
    content((rel: (0.15, 0.1), to: "box.north-west"), anchor: "west",
      box(fill: orange, radius: 2pt, inset: 3pt, title))
    on-layer(-1, line(p, (p, "|-", a), a))
    circle(p, radius: 2pt, fill: black)
  }

  for e in (
    (1786, (0, 1.9), -170deg, [1786 - William Playfair],
      [Line, bar and pie charts.]),
    (1854, (0, -2.5), 140deg, [1854 - John Snow],
      [Mapping that found \ the source of cholera.]),
    (1858, (0, 0.5), -168.5deg, [1858 - Florence Nightingale],
      ["Coxcomb" diagrams of the British army.]),
    (1861, (0, -0.8), 158deg, [1861 - Charles Minard],
```

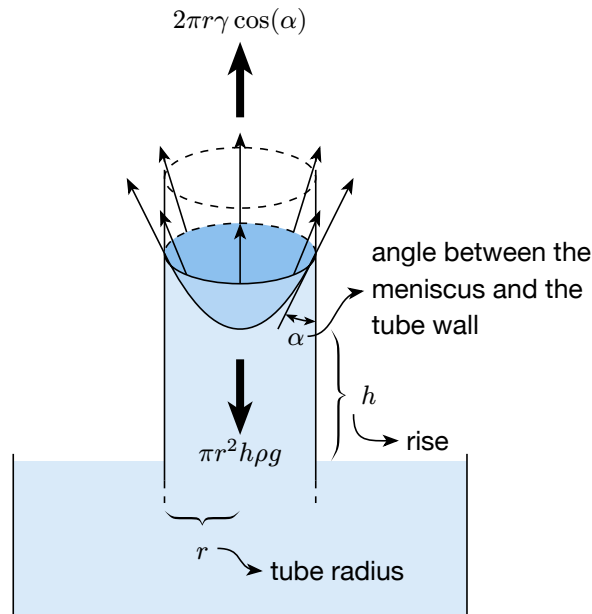
```

[Napoleon's army \ marches on Russia.]),
(1914, (-0.2, 3.8), 0deg, [1914 - Willard Brinton],
[_Graphic methods for presenting facts_ \ Visualization for business.]),
(1952, (0, -0.8), 30deg, [1952 - Mary Eleanor Spear],
[_Charting statistics_ \ Good practice in the \ US government.]),
(1967, (0, 1.0), -16deg, [1967 - Jacques Bertin],
[_Sémiologie graphique_ \ Theory of vis. and 7 visual variables.]),
(1970, (0, -3.0), 24deg, [1970s - John Tukey],
[Vis. with computers: \ exploratory and confirmatory.]),
(1983, (0, 3.7), -20deg, [1983 - Edward Tufte],
[_The visual display of quantitative information_ \ Statistical rigour, clarity,
design.]),
(1984, (0, -5.2), 90deg, [1984 - W. Cleveland & R. McGill],
[Measuring graphical perception and effective vis.]),
(1986, (0.2, 2.8), 180deg, [1986 - Jock Mackinlay],
[Thesis on J. Bertin for the digital age.]),
(1999, (0, -3.0), 150deg, [1999 - Leland Wilkinson],
[_The grammar of graphics_ \ A concise grammar for \ graphical components.]),
(2010, (0, 0.7), -90deg, [2010 - Ronald Rensink],
[Perception · Weber's law \ Graphical effectiveness.]),
(2018, (0, -0.8), 150deg, [Today],
[Tools for building vis. \ Interactive and real-time vis. \ Dashboards]),
) { callout(..e) }
})

```

Capillary rise

Redrawn from [bouma.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*. A pseudo-3D tube from elliptical arcs; arrows at elliptical polar coordinates; `cetz.angle.angle` for the contact angle; brace for dimensions; curved callouts.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetz.canvas({
  import cetz.draw: *
  import cetz.decorations: brace
  set-style(stroke: .6pt + black, mark: (fill: black, scale: .8), brace: (amplitude: .2,
content-offset: .1),
    content: (padding: .1))
  let arrow = line.with(mark: (end: "stealth"))
  let callout(from, to, body, out: 0deg) = {
    bezier(from, to, (rel: (out, .4), to: from), (rel: (-.4, 0), to: to), mark: (end:
"stealth"))
    content(to, body, anchor: "west")
  }
  let water = blue.lighten(85%)
  // Tube radius and its foreshortened depth; meniscus contact height and dip; tube top;
  // pool surface and floor.
  let (r, ry, m, dip, top, level, floor) = (1, .4, 3, 1, 4, .25, -1.8)

  rect((-3 * r, floor), (3 * r, level), fill: water, stroke: none)
  line((-3 * r, level + .1), (-3 * r, floor), (3 * r, floor), (3 * r, level + .1))
  rect((-r, 0), (r, m), fill: water, stroke: none)

  for (side, x) in (("left", -r), ("right", r)) {
    line((x, 0), (x, top), name: side)
    line((x, 0), (rel: (0, -.3)), stroke: (dash: "dashed"))
    line((x, top), (rel: (0, .3)), stroke: (dash: "dashed"))
  }
  circle((0, top), radius: (r, ry), stroke: (dash: "dashed"))

  // A quadratic Bézier is an exact parabola; its control point lies on both contact tangents.
  anchor("ctrl", (0, m - 2 * dip))
  anchor("surface", (0, m))
  anchor("contact", (r, m))
  let rim = arc.with("surface", radius: (r, ry), anchor: "origin", start: 0deg)
```

```

circle("surface", radius: (r, ry), fill: blue.lighten(55%), stroke: none)
rim(delta: 180deg, stroke: (dash: "dashed"))
merge-path(fill: blue.lighten(70%), {
  rim(delta: -180deg)
  bezier((-r, m), "contact", "ctrl")
})

// Surface tension acts along the contact tangent, all around the rim.
for a in range(0, 360, step: 45) {
  let a = a * 1deg
  arrow((rel: (a, (r, ry)), to: "surface"), (rel: (.5 * calc.cos(a), 1 + .2 * calc.sin(a))))
}
line("contact", ("contact", 50%, "ctrl"))
cetz.angle.angle("contact", "right.start", "ctrl", direction: "near", radius: .9,
  label: $alpha$, label-radius: 125%, mark: (symbol: "stealth", scale: .5),
  name: "alpha")
callout((rel: (.15, 0), to: "alpha.label"), (rel: (.9, .6), to: "alpha.label"),
  [angle between the \ meniscus and the \ tube wall])

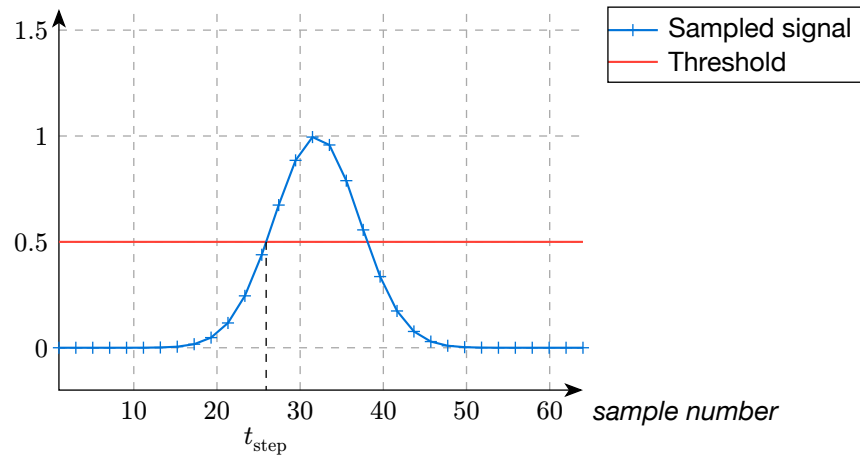
arrow((0, m - dip - .4), (rel: (0, -1)), stroke: 3pt, name: "weight")
content("weight.end", $pi r^2 h rho g$, anchor: "north")
arrow((0, top + .8), (rel: (0, 1)), stroke: 3pt, name: "lift")
content("lift.end", $2 pi r gamma cos(alpha)$, anchor: "south")

brace((r + .2, m - dip), (r + .2, level), name: "h")
content("h.content", $h$, anchor: "west")
callout((rel: (0, -.25), to: "h.content"), (rel: (.6, -.6), to: "h.content"), [rise],
  out: -90deg)
brace((0, -.4), (-r, -.4), name: "r")
content("r.content", $r$, anchor: "north")
callout((rel: (.2, -.2), to: "r.content"), (rel: (.8, -.5), to: "r.content"), [tube radius])
}

```

Threshold crossing

Redrawn from [anotacoes_intersecao.png](#) in Walmes M. Zeviani's *Tikz Gallery*, which is based on [an answer by Red](#) on TeX Stack Exchange (CC BY-SA 3.0). `intersections` finds where the signal crosses the threshold, and a `"|-"` coordinate drops it to a computed tick; standalone marks on data points; a legend boxed with `rect-around`.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// Based on https://tex.stackexchange.com/a/130791 by Red, CC BY-SA 3.0.

#cetx.canvas({
  import cetx.draw: *
  set-style(stroke: .5pt + black, content: (padding: 4pt))
  // Data coordinates (sample, amplitude); text, strokes and marks keep their size.
  scale(x: .11, y: 2.8)

  let (x0, x1, y0, y1) = (1, 64, -.2, 1.6)
  let signal(x) = calc.exp(-calc.log(2) / 16 * calc.pow(x - 32, 2))
  let samples = range(32).map(i => x0 + (x1 - x0) * i / 31).map(x => (x, signal(x)))

  for x in range(10, 61, step: 10) {
    line((x, y0), (x, y1), stroke: (paint: gray, dash: "dashed"))
    content((x, y0), $#x$, anchor: "north")
  }
  for y in (0, .5, 1, 1.5) {
    line((x0, y), (x1, y), stroke: (paint: gray, dash: "dashed"))
    content((x0, y), $#y$, anchor: "east")
  }
  line((x0, y1), (x0, y0), (x1, y0), mark: (start: "stealth", end: "stealth", fill: black))
  content((x1, y0), emph[sample number], anchor: "north-west")

  line((x0, .5), (x1, .5), stroke: .8pt + red, name: "threshold")
  let plus(p) = mark(p, 0deg, "+", stroke: blue, length: .16, width: .16)
  line(..samples, stroke: .8pt + blue, name: "signal")
  samples.map(plus).join()

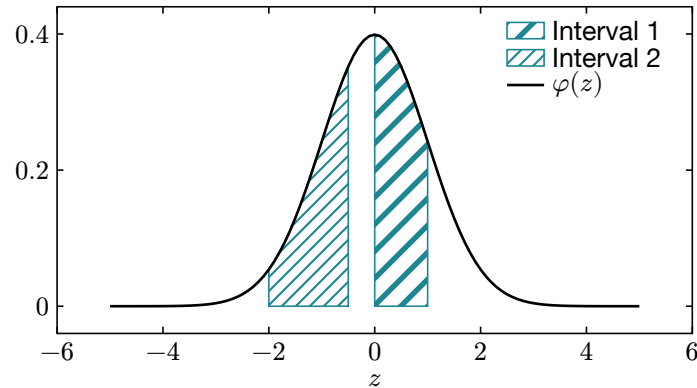
  // "cross.0" is the rising crossing, the first one along the signal.
  intersections("cross", "signal", "threshold")
  line("cross.0", ("cross.0", "|-", (0, y0)), stroke: (dash: "dashed"), name: "drop")
  content((rel: (0, -.12), to: "drop.end"), $t_"step"$, anchor: "north")

  group(name: "legend", {
    set-origin((x1 + 3, y1))
    for (i, (hue, label)) in ((blue, [Sampled signal]), (red, [Threshold])).enumerate() {
      line((0, -.15 * i - .1), (rel: (6, 0)), stroke: .8pt + hue, name: "key")
    }
  })
})
```

```
    if i == 0 { plus("key.50%") }  
    content("key.end", label, anchor: "west")  
  }  
})  
rect-around("legend")  
})
```

Hatched intervals

Redrawn from [preenchimento.pgfc \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*, which is based on [an answer by Jake](#) on TeX Stack Exchange (CC BY-SA 3.0). Typst tiling hatches with spacing and thickness as parameters; data units via scale; one loop for each series and its legend row.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)
// Based on https://tex.stackexchange.com/a/29367 by Jake, CC BY-SA 3.0.

#let hue = eastern.darken(15%)
// Diagonal hatch with period `d`; lines overshoot the tile so that butt caps do not notch the
// corners.
#let hatch(d, thickness) = tiling(size: (d, d), for c in (0pt, d, 2 * d) {
  place(line(start: (c + d, -d), end: (-d, c + d), stroke: thickness + hue))
})

#cetx.canvas({
  import cetx.draw: *
  set-style(stroke: .6pt + black)
  // Data coordinates (z, phi(z)).
  scale(x: .7, y: 9)

  let pdf(z) = calc.exp(-z * z / 2) / calc.sqrt(2 * calc.pi)
  let curve(a, b) = range(101).map(i => a + (b - a) * i / 100).map(z => (z, pdf(z)))

  rect((-6, -.04), (6, .44), name: "frame")
  for x in range(-6, 7, step: 2) {
    let p = ((x, 0), "|-", "frame.south")
    line(p, (rel: (0, .01)))
    line((p, "|-", "frame.north"), (rel: (0, -.01)))
    content(p, $x$, anchor: "north", padding: 3pt)
  }
  for y in (0, .2, .4) {
    let p = ("frame.west", "|-", (0, y))
    line(p, (rel: (.12, 0)))
    line((p, "-|", "frame.east"), (rel: (-.12, 0)))
    content(p, $y$, anchor: "east", padding: 3pt)
  }
  content((rel: (0, -.05), to: "frame.south"), $z$, anchor: "north")

  // Each series with its legend row.
  for (i, (fill, label, (a, b))) in (
    (hatch(10pt, 2pt), [Interval 1], (0, 1)),
    (hatch(5pt, .75pt), [Interval 2], (-2, -.5)),
    (none, $phi(z)$, (-5, 5)),
  ).enumerate() {
    let p = (rel: (-2.8, -.035 - .04 * i), to: "frame.north-east")
    if fill == none {
      line(..curve(a, b), stroke: 1pt)
    }
  }
})
```

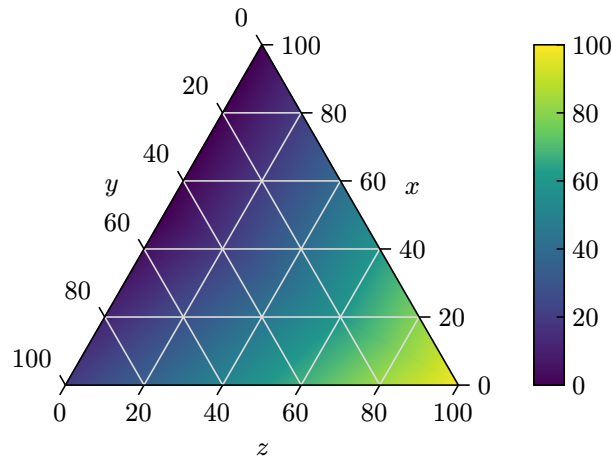
```

    line((rel: (-.7, 0), to: p), p, stroke: 1pt)
  } else {
    line((a, 0), ..curve(a, b), (b, 0), close: true, fill: fill, stroke: hue)
    rect((rel: (-.7, -.012), to: p), (rel: (0, .012), to: p), fill: fill, stroke: hue)
  }
  content(p, label, anchor: "west", padding: 3pt)
}
})

```

Ternary diagram

Redrawn from [ternario.pdf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*, which is based on [an answer by Christian Feuersänger](#) on TeX Stack Exchange (CC BY-SA 3.0). Barycentric coordinates; each triangle filled with an exact gradient.linear sampled from `color.map.viridis`; one loop for the grid, ticks and labels of all three sides; a matching color bar.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)
// Based on https://tex.stackexchange.com/a/277000 by Christian Feuersänger, CC BY-SA 3.0.

#let cmap = gradient.linear(..color.map.viridis, angle: -90deg)
#let keys = ("x", "y", "z")
#let corners = range(3).map(i => 90deg + i * 120deg).map(a => (3 * calc.cos(a), 3 *
calc.sin(a)))

#cetz.canvas{
  import cetz.draw: *
  import cetz: vector
  set-style(stroke: .6pt + black, content: (padding: 3pt))
  for (k, c) in keys.zip(corners) { anchor(k, c) }

  // Gouraud-shade a triangle given as rows (x, y, z, value) of the source table.
  let patch(..rows) = {
    let rows = rows.pos()
    let pts = rows.map(r => corners.zip(r).map((c, w)) => vector.scale(c,
w)).reduce(vector.add))
    let vs = rows.map(r => r.last())
    // Gradient of the linear interpolant, from the sides rotated a quarter turn.
    let normals = range(3).map(i => vector.sub(pts.at(calc.rem(i + 1, 3)), pts.at(calc.rem(i +
2, 3))))
    .map(((dx, dy)) => (dy, -dx))
    let det = range(3).map(i => pts.at(i).at(0) * normals.at(i).at(0)).sum()
    let g = range(3).map(i => vector.scale(normals.at(i), vs.at(i) / det)).reduce(vector.add)
    // Typst turns gradients clockwise and stretches them between the extreme bounding-box
    corners.
    let (xs, ys) = array.zip(..pts)
    let vc = for x in (calc.min(..xs), calc.max(..xs)) { for y in (calc.min(..ys),
calc.max(..ys)) {
      (vs.first() + vector.dot(g, vector.sub((x, y), pts.first())),)
    } }
    let (lo, hi, va, vb) = (calc.min(..vs), calc.max(..vs), calc.min(..vc), calc.max(..vc))
    let stops = range(11).map(k => lo + (hi - lo) * k / 10)
    .map(v => (cmap.sample(v * 1%), (v - va) / (vb - va) * 100%))
    let paint = gradient.linear((stops.first().at(0), 0%), ..stops, (stops.last().at(0), 100%),
angle: -calc.atan2(..g))
    line(..pts, close: true, fill: paint, stroke: .5pt + paint)
```

```

}
patch((0, 0, 1, 100), (1, 0, 0, 0), (.5, .5, 0, 0))
patch(.5, .5, 0, 0), (0, 1, 0, 20), (0, 0, 1, 100))

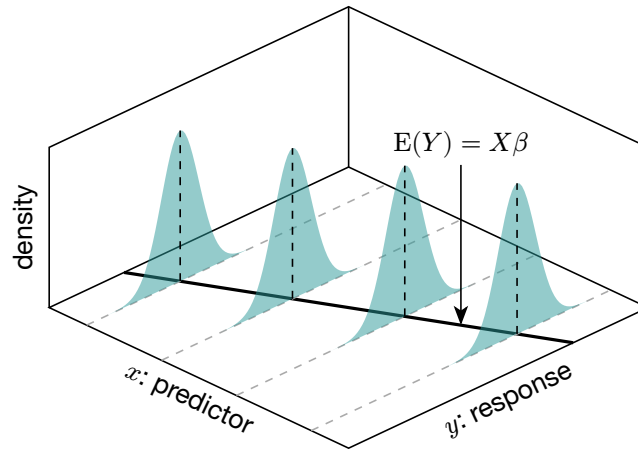
// Axis k is read along the side where the next key is zero; its lines of constant k
// run from that side towards the side where the key after next is zero.
for (i, k) in keys.enumerate() {
  let (a, b) = (keys.at(calc.rem(i + 1, 3)), keys.at(calc.rem(i + 2, 3)))
  let dir = i * 120deg
  for v in range(0, 101, step: 20) {
    let t = v / 100
    let edge = (bary: ((k): t, (b): 1 - t))
    if 0 < v and v < 100 { line((bary: ((k): t, (a): 1 - t)), edge, stroke: silver) }
    line(edge, (rel: (dir, .15)), name: "tick")
    content("tick.end", $v$, anchor: ("west", "south-east", "north").at(i))
  }
  content((rel: (dir + 30deg, .8), to: (bary: ((k): 1, (b): 1))), $italic(#k)$)
}
line(..keys, close: true)

rect((rel: (1, 0), to: "z"), ((rel: (1.4, 0), to: "z"), "|-", "x"), fill: cmap,
  name: "bar")
for v in range(0, 101, step: 20) {
  let p = ("bar.south-east", v * 1%, "bar.north-east")
  line(p, (rel: (-.1, 0)))
  content(p, $v$, anchor: "west")
}
})

```

Regression densities in 3D

Redrawn from [reg_model_3d.pgf \(png\)](#) in Walmes M. Zeviani's *Tikz Gallery*, which is based on [an answer by Jake](#) on TeX Stack Exchange (CC BY-SA 3.0). An ortho view drawn back to front; on-xz for the floor and on-xy for density curves standing on it; labels rotated to the projected edges.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)
// Based on https://tex.stackexchange.com/a/53795 by Jake, CC BY-SA 3.0.

#let normal(y, mu) = calc.exp(-calc.pow(y - mu, 2) / 2) / (2 * calc.sqrt(calc.pi))

#cezt.canvas({
  import cetz.draw: *
  let elev = 28deg
  // Screen slope of the floor edges, used to lay labels along them.
  let tilt = calc.atan(calc.sin(elev))
  let (lo, hi, top) = (-4, 12, 0.3)

  // Canvas x = response y, canvas z = predictor x, canvas y = density.
  ortho(x: elev, y: -45deg, sorted: false, {
    scale(x: 0.35, y: 8, z: 0.7)
    set-style(stroke: 0.6pt + black)

    on-xz(rect((lo, 0), (hi, 8)))
    line((lo, 0, 0), (lo, top, 0), (hi, top, 0), (hi, top, 8), (hi, 0, 8))
    line((hi, 0, 0), (hi, top, 0))
    on-xz(line((0, 0), (8, 8), stroke: 1.2pt))

    for x in (1, 3, 5, 7) {
      on-xz(line((lo, x), (hi, x), stroke: (paint: gray, dash: "dashed")))
      on-xy(z: x, {
        let ys = range(61).map(i => x - 4 + i * 8 / 60)
        line(..ys.map(y => (y, normal(y, x))), close: true, stroke: none,
          fill: teal.darken(25%).transparentize(50%))
        line((x, 0), (x, normal(x, x)), stroke: (dash: "dashed"))
      })
    }

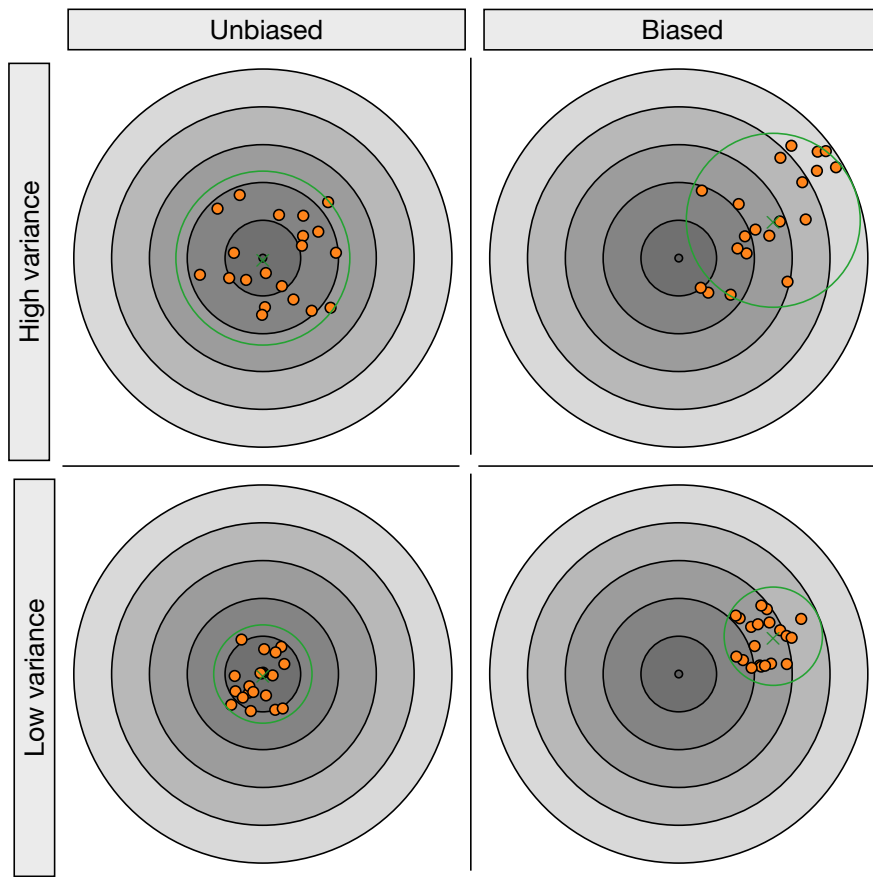
    on-xy(z: 6, {
      line((6, top), (6, 0), mark: (end: "stealth", fill: black))
      content((6, top), $supright(E)(Y) = X beta$, anchor: "south", padding: 0.1)
    })

    content((lo, 0, 4), [$x$: predictor], angle: -tilt, anchor: "north", padding: 0.15)
    content((4, 0, 8), [$y$: response], angle: tilt, anchor: "north", padding: 0.15)
    content((lo, top / 2, 0), [density], angle: 90deg, anchor: "south", padding: 0.15)
  })
}
```

```
}  
}
```

Bias and variance targets

Redrawn from [bias-variance.png](#) in Walmes M. Zeviani's *Tikz Gallery*. Small multiples from one parameterized panel in nested loops; seeded pseudo-random hits from a tiny inline generator; rotated row headers.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// Linear congruential generator mapped to [-1, 1); Typst has no random numbers.
#let uniform(seed, n) = {
  let m = calc.pow(2, 31)
  let s = seed
  for _ in range(n) {
    s = calc.rem(1103515245 * s + 12345, m)
    (2 * s / m - 1,)
  }
}

#cetz.canvas({
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)

  let panel(name, origin, aim, spread, ring, seed) = group(name: name, {
    translate(origin)
    for r in (2.5, 2, 1.5, 1, 0.5, 0.05) {
      circle((0, 0), radius: r, fill: black.transparentize(85%))
    }
    for (dx, dy) in uniform(seed, 40).chunks(2) {
      circle((rel: (spread * dx, spread * dy), to: aim), radius: 2pt, fill: orange)
    }
    circle(aim, radius: ring, stroke: green.darken(20%))
    content(aim, text(green.darken(30%), $times$))
  })
```

```

    line((-2.65, -2.75), (rel: (5.25, 0)))
    line((2.75, 2.65), (rel: (0, -5.25)))
  })
  let header(pos, body, ..args) = content(pos, ..args, padding: 0.1,
    box(width: 5.25cm, inset: 4pt, fill: silver.lighten(40%), stroke: 0.6pt, align(center,
body)))

  let cols = (([Unbiased], (0, 0)), ([Biased], (1.25, 0.5)))
  let rows = (([High variance], 1, 1.15), ([Low variance], 0.5, 0.65))
  for (j, (row, spread, ring)) in rows.enumerate() {
    for (i, (col, aim)) in cols.enumerate() {
      let name = "p" + str(i) + str(j)
      panel(name, (5.5 * i, -5.5 * j), aim, spread, ring, 1 + 2 * j + i)
      if i == 0 { header(name + ".west", row, angle: 90deg, anchor: "south") }
      if j == 0 { header(name + ".north", col, anchor: "south") }
    }
  }
}
})

```